IMT Nord Europe
École Mines-Télécom
IMT-Université de Lille

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

Projet IoT

Capteur de température et d'humidité « TempHumiX »

Simulation, communication MQTT et IHM d'archivage

José Manuel VILLA ROMERO

IMT Nord Europe – M2, UV IoT / IIoT
Poste n° 42906

Enseignant responsable : Abdellah BOUCHAMA
Enseignant : Bassem BEN HASSEN

Dépôt GitHub : `https://github.com/jmvilla12/IOT_JMVR`

Novembre 2026

Table des matières

Introduction **2**

1 Partie 1 : simulation sous SimulIDE **2**
1.1 Organisation . . . 2
1.2 Les mini projets . . . 2
1.3 Le capteur TempHumiX . . . 3

2 Partie 2 : communication MQTT avec le boîtier réel **4**
2.1 Paramètres et topics . . . 4
2.2 Premiers tests avec MQTTX . . . 5
2.3 Le programme du capteur . . . 6
2.4 Analyse des trames avec Wireshark . . . 8
2.5 QoS, messages retenus et jokers . . . 10
2.6 Questions générales . . . 11

3 Partie 3 : IHM d'acquisition et d'archivage **12**
3.1 Architecture . . . 12
3.2 Clients MQTT en console . . . 12
3.3 Interface PySide6 . . . 13
3.4 Archivage dans SQLite . . . 14
3.5 Affichage des courbes . . . 15
3.6 Notice de fonctionnement . . . 15
3.7 Le challenge . . . 16

Conclusion **16**

Introduction

Le but du projet est de construire un capteur connecté de température et d'humidité, appelé TempHumiX, puis l'application qui va avec. Le travail s'est fait en trois étapes :

1. la conception du capteur en simulation sous SimulIDE, avec un Arduino UNO, un DHT22 et un écran LCD ;
2. le passage sur le boîtier réel (Arduino UNO R4 WiFi) et la communication en MQTT avec le broker de la salle ;
3. une IHM en Python (PySide6) qui affiche les mesures, envoie des commandes au boîtier, archive les données dans SQLite et trace les courbes.

Tout le code est versionné sur GitHub, avec un commit par étape. Dans tout le rapport, l'identifiant de mon poste est **42906** : c'est lui qui termine tous les topics MQTT.

1 Partie 1 : simulation sous SimulIDE

1.1 Organisation

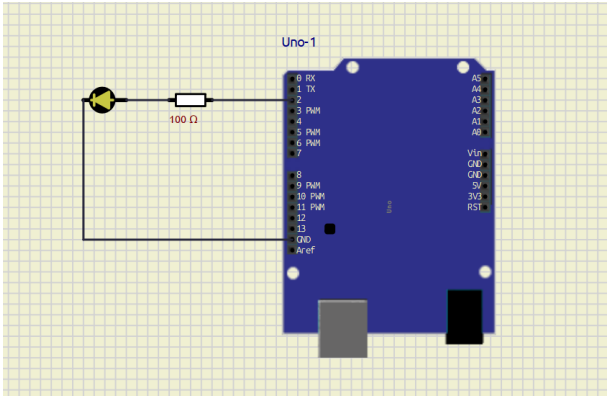
Chaque mini projet a son propre dossier dans `SIMULIDE/`, avec le circuit (`.sim1`) et le code (`.ino`) qui portent le même nom, en minuscules et avec des « _ ». Pour tester un programme, je le compile dans l'IDE Arduino (*Export Compiled Binary*) et je charge le `.hex` dans l'Arduino de SimulIDE.

1.2 Les mini projets

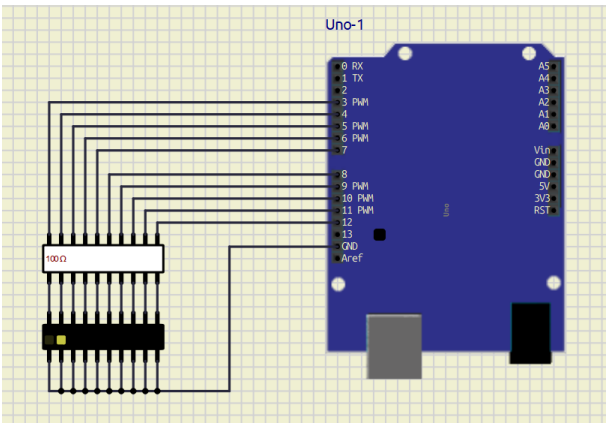
J'ai fait les circuits dans l'ordre du sujet, chacun avec un seul nouveau composant à la fois :

— `clignotement_led` : une LED sur la broche 2 qui clignote (500 ms allumée, 500 ms éteinte) ;
— `bargraphe` : un chenillard sur un bargraphe de 10 LEDs (broches 3 à 12), une LED toutes les 100 ms ;
— `capteur_dht22` : le DHT22 sur A0, une LED verte si la température est en dessous de 20 °C et une rouge sinon ;
— `lcd_i2c` : l'afficheur AIP31068 en I2C (adresse 0x3E) qui affiche « HELLO / WORLD » ;
— `lcd_i2c_dht22` : le LCD et le DHT22 ensemble, avec « IMT NE - PPI » et la ligne température / humidité.

Pour le LCD, il faut faire attention au nom de la bibliothèque : le fichier à inclure est `LiquidCrystal_AIP31068_I2C.` et pas `LiquidCrystal_I2C.h` comme dans la documentation.



(a) Clignotement de la LED



(b) Chenillard sur le bargraphe

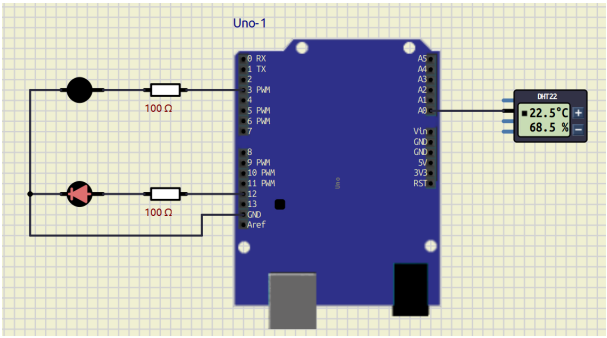


FIGURE 2 – DHT22 et LEDs de seuil

Que peut-on conclure des différents circuits ? Chaque circuit valide un seul élément : une sortie simple, plusieurs sorties, le capteur, puis l'afficheur. Quand on arrive au capteur complet, chaque partie a déjà été testée toute seule, et s'il y a un problème on sait plus vite où chercher. J'ai aussi gardé les mêmes broches d'un circuit à l'autre (LED de vie sur 2, bargraphe sur 3 à 12, DHT22 sur A0), donc le code de TempHumiX est surtout un assemblage des programmes précédents.

1.3 Le capteur TempHumiX

Le circuit final reprend tous les composants : Arduino UNO, DHT22, LCD 16x2 I2C, LED de vie, bargraphe et port série. Le programme doit faire plusieurs choses à des rythmes différents : faire clignoter la LED de vie à 2 Hz, lire le DHT22 toutes les 2 s, faire avancer un dateur (avec des mois de 30 jours pour simplifier) et afficher date, température et humidité sur le LCD et sur le port série.

Pour ça j'ai suivi le conseil du sujet : une machine d'état avec un `delay(100)` dans la `loop`. Un compteur `tick` augmente de 1 à chaque passage, donc chaque tâche se déclenche avec un simple modulo :

Tâche	Période	Condition
LED de vie (2 Hz)	500 ms	`tick % 5`
Dateur et ligne 1 du LCD	1 s	`tick % 10 == 0`
Lecture DHT22, ligne 2 du LCD, port série	2 s	`tick % 20 == 0`

TABLE 1 – Tâches de la machine d'état (1 tick = 100 ms)

```
void loop()
{
  // LED de vie à2 Hz : période 500 ms = 5 ticks
  digitalWrite(PIN_LED_VIE, (tick % 5) < 2 ? HIGH : LOW);

  if (tick % 10 == 0) avancerDate(); // +1 seconde
  if (tick % 20 == 0) { /* lecture DHT22, LCD ligne 2, port série */ }
  if (tick % 10 == 0) { /* LCD ligne 1 : date et heure */ }

  tick++;
  delay(100);
}
```

Le bargraphe affiche l'humidité (10 % par LED). Comme `sprintf` ne gère pas les `float` sur Arduino, je passe par `dtostrf` pour avoir une décimale.

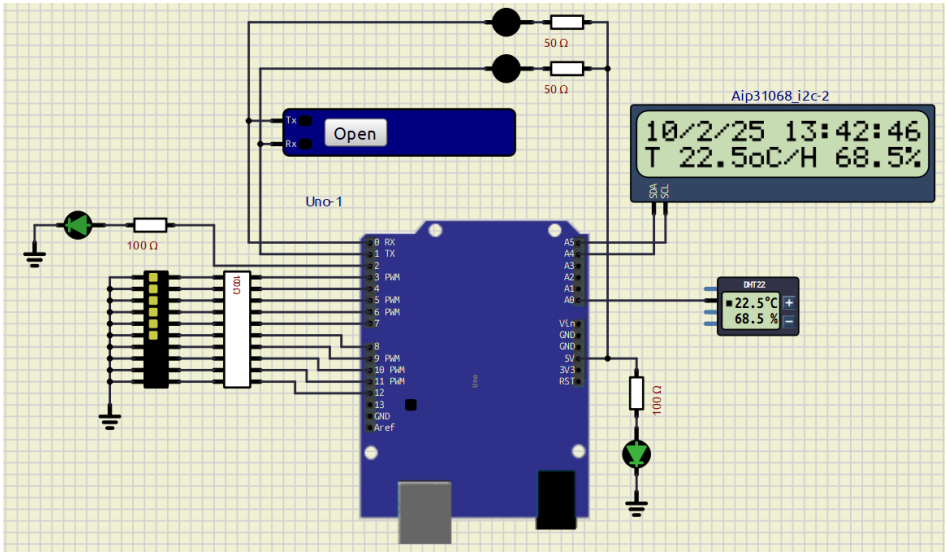


FIGURE 3 – Circuit TempHumiX en simulation

2 Partie 2 : communication MQTT avec le boîtier réel

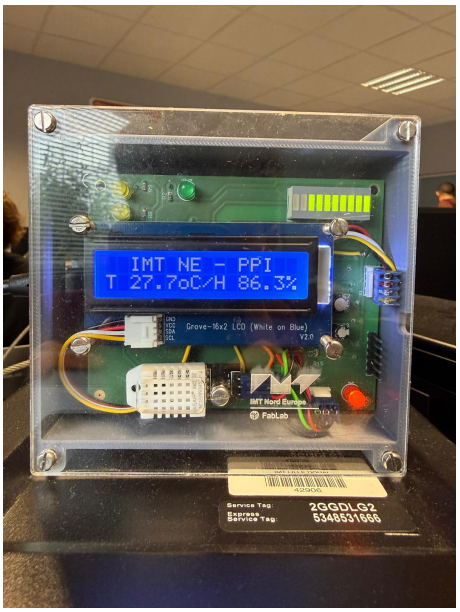
2.1 Paramètres et topics

Le boîtier et le PC sont sur le WiFi « IoT IMT Nord Europe », un réseau sans accès Internet. Le broker Mosquitto est à l'adresse 192.168.1.1, port 1883, avec l'utilisateur `mos_user`. Les topics sont construits avec le groupe `/iot/`, le nom de la donnée et le numéro du poste :

Sens	Topic	Contenu
Publié	`/iot/VERSION_42906`	version du programme (1.0)
Publié	`/iot/TEMPERATURE_42906`	température, au dixième de °C
Publié	`/iot/HUMIDITY_42906`	humidité, au dixième de %
Publié	`/iot/GET_DATE_42906`	date `JJ/MM/AAAA HH:MM:SS`
Souscrit	`/iot/COMMAND_42906`	code commande (10 à 60)
Souscrit	`/iot/BARGRAPH_42906`	valeur de 0 à 1023
Souscrit	`/iot/BLINK_42906`	période de clignotement en ms

TABLE 2 – Topics échangés avec le boîtier

Les codes commande sont : 10 synchronisation NTP, 20 démarrage de la mesure, 30 arrêt, 40 clignotement lent, 50 rapide et 60 clignotement personnalisé (avec la période reçue sur `BLINK_`). Le numéro 42906 est celui de l'étiquette collée sur mon poste.



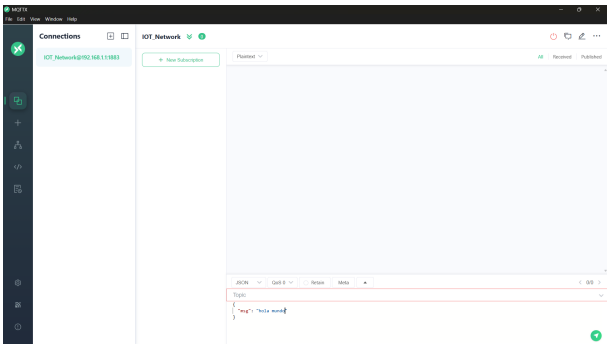
(a) Le boîtier réel après le passage à l'UNO R4



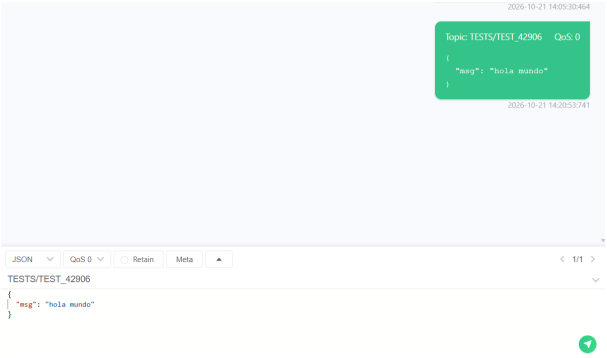
(b) Étiquette du poste : 42906

2.2 Premiers tests avec MQTTX

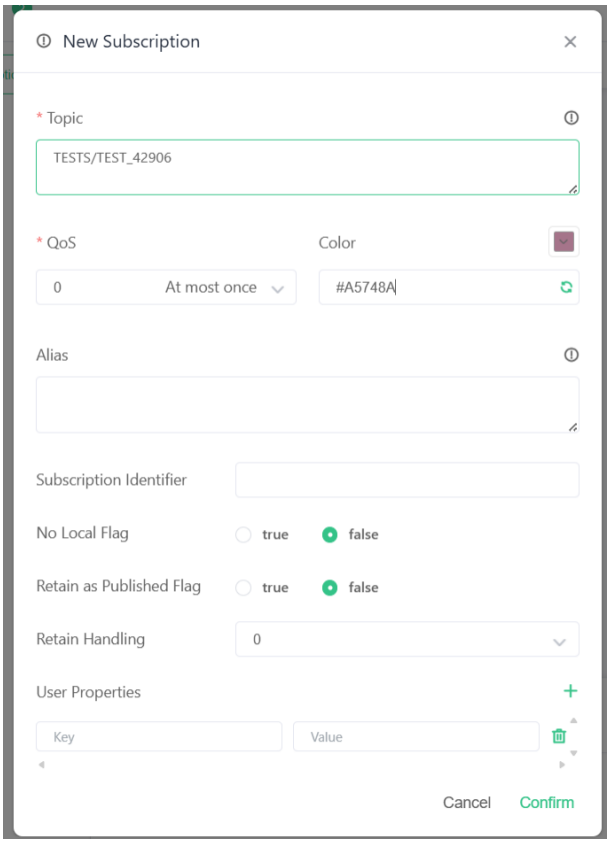
Avant de coder quoi que ce soit, j'ai vérifié le broker à la main avec MQTTX : connexion, publication sur `TESTS/TEST_42906`, puis souscription au même topic. Le message publié revient bien, ce qui montre que le broker renvoie aussi les messages à leur propre émetteur quand il est abonné.



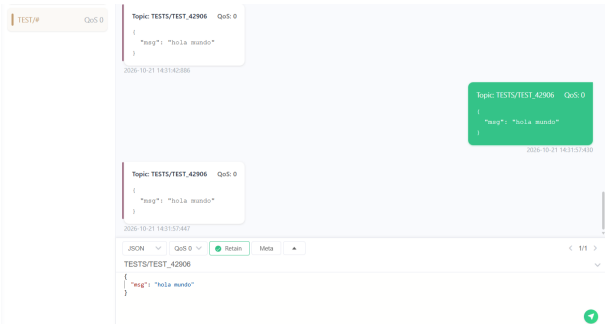
(a) Connexion au broker



(b) Publication sur le topic de test



(a) Création de la souscription



(b) Souscription avec joker et Retain

Ce que j'ai observé en jouant avec MQTTX :

— en m'abonnant au topic d'un autre poste, je reçois ses messages. Rien n'isole les clients : tout le monde peut lire et écrire sur `TESTS/` ;

— avec `TESTS/#`, je reçois les messages de tous les postes, car # remplace tous les niveaux qui suivent ;

— avec *Retain*, le broker garde le dernier message du topic et l'envoie tout de suite à chaque nouvel abonné, même s'il se connecte après la publication.

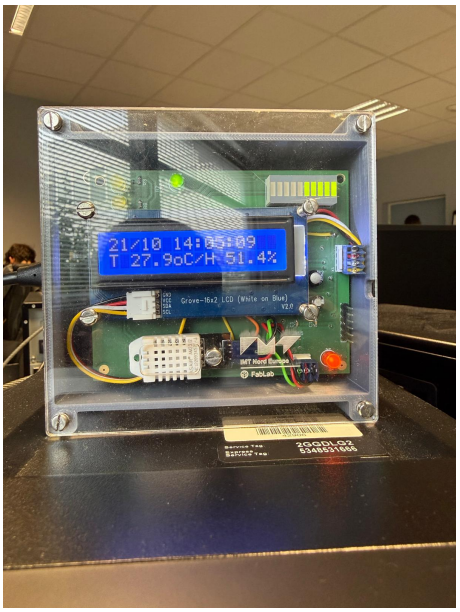
2.3 Le programme du capteur

J'ai repris le code de la simulation et je l'ai fait évoluer par petites étapes, avec un commit à chaque fois. Ça permet de voir dans l'historique ce qui change à chaque partie du sujet :

Commit	Changement
6.1.1	Passage à l'UNO R4 : bibliothèque Grove LCD (`rgb_lcd`), suppression du dateur logiciel
6.1.2	Date et heure lues sur le RTC interne du R4
6.1.3 a-d	Connexion WiFi (`setup_wifi()`), écran d'accueil avec version et adresse IP
6.1.3 e	Mise à l'heure du RTC par NTP, message « Pas de synch NTP » si ça échoue
6.1.4	Connexion au broker (`setup_mqtt()`) et publication des 4 topics toutes les 2 s
6.1.5	Souscription à COMMAND, BARGRAPH et BLINK avec la fonction `onMqttMessage()`
6.1.6	Version publiée seulement après la commande 10, mesures publiées seulement si elles changent

TABLE 3 – Évolution du programme du boîtier

J'ai gardé la machine d'état à 100 ms au lieu des 500 ms proposés, parce que la période envoyée sur `BLINK_` doit être un multiple de 100 ms. La période de la LED est donc une variable en ticks, que les commandes 40, 50 et 60 modifient.



(a) 6.1.2 : date et heure lues sur le RTC



(b) 6.1.3 : écran d'accueil, version et adresse IP

WiFi en 5 GHz. Quand j'essaie de me connecter au réseau « IoT IMT Nord Europe 5G », la connexion échoue toujours et l'écran affiche « WiFi KO ». Le module ESP32-S3 de l'UNO R4 ne fonctionne qu'en 2,4 GHz, il ne voit donc pas ce réseau.

NTP. Au début j'utilisais le serveur NTP de l'école (10.12.130.1), mais le réseau IoT n'a pas de sortie : l'écran affichait « Pas de synch NTP » et la date publiée restait au 01/01/2000 (on le voit sur la figure 9). Le programme n'est pas bloqué pour autant, la température et l'humidité continuent de s'afficher. En testant les adresses du réseau avec `w32tm`, j'ai trouvé un serveur NTP qui répond en 192.168.1.254. J'ai aussi corrigé le décalage horaire : 3600 s depuis le passage à l'heure d'hiver.

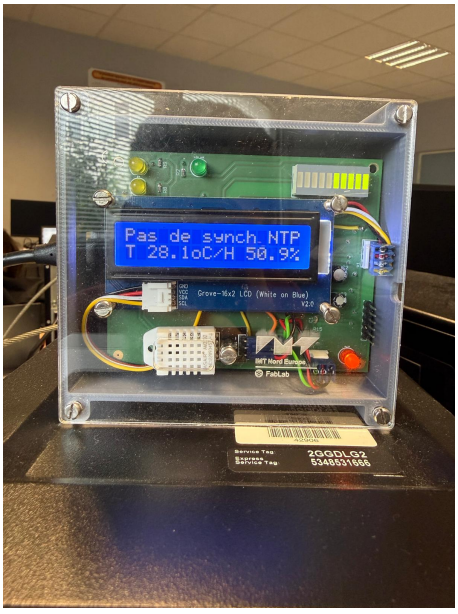


FIGURE 8 – Sans serveur NTP joignable, le message s'affiche mais les mesures continuent

Les échanges avec le boîtier. Au démarrage, le boîtier se connecte au WiFi, se met à l'heure, puis envoie un CONNECT au broker avec l'identifiant `IdO_Client_42906` et s'abonne à ses trois topics de commande.

Ensuite, toutes les 2 s, il lit le DHT22 et, si la température ou l'humidité a changé, il publie dans l'ordre la température, l'humidité et la date. Quand un message arrive, `mqtt.poll()` appelle le callback :

```
void onMqttMessage(int tailleMessage)
{
  String topic = mqtt.messageTopic();
  String message = "";
  while (mqtt.available()) message += (char)mqtt.read();
  int valeur = message.toInt();

  if (topic.indexOf("COMMAND") >= 0) {
    if (valeur == 10 && synchroNTP()) publier(T_VERSION, VERSION);
    if (valeur == 20) mesureActive = true;
    if (valeur == 30) mesureActive = false;
    if (valeur == 40) periodeLed = 10; // lent
    if (valeur == 50) periodeLed = 2; // rapide
    if (valeur == 60) periodeLed = periodePerso;
  }
  else if (topic.indexOf("BARGRAPH") >= 0) afficherBargraphe(valeur);
  else if (topic.indexOf("BLINK") >= 0) periodePerso = max(2, valeur / 100);
}
```

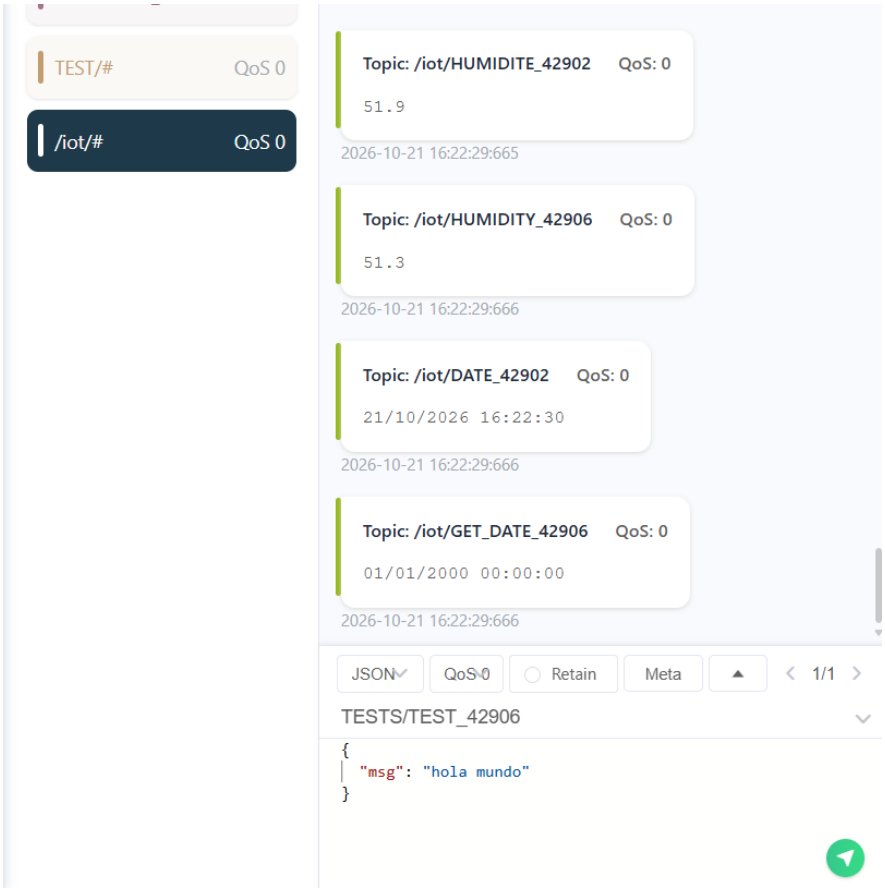


FIGURE 9 – Topics reçus dans MQTTX avec l'abonnement `/iot/#`

2.4 Analyse des trames avec Wireshark

J'ai capturé une session MQTTX complète (connexion, souscription, publication, déconnexion) avec le filtre de capture `tcp port 1883`, puis le filtre d'affichage `mqtt`.

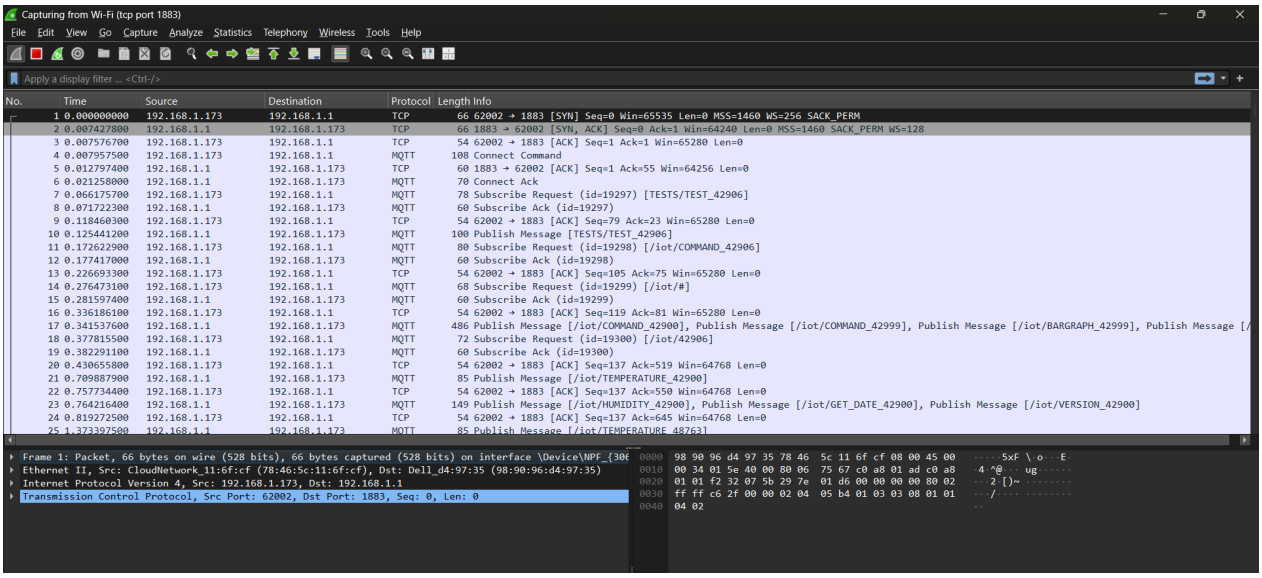


FIGURE 10 – Début de session : Connect, Subscribe, Publish

Paquet	Sens	Ce que j'ai relevé
CONNECT	client → broker	MQTT v5.0, Client ID `mqttx_26a1553c`, Keep Alive 60 s, User Name et Password `mos_user`
CONNACK	broker → client	code retour 0, connexion acceptée
SUBSCRIBE	client → broker	topic `TESTS/Test_42906`, QoS 0 demandé
SUBACK	broker → client	QoS 0 accordé
PUBLISH	les deux sens	topic, message, QoS 0, Retain et DUP à 0
PINGREQ / PINGRESP	client ↔ broker	envoyés quand il n'y a pas eu d'échange pendant le Keep Alive
DISCONNECT	client → broker	fin de session demandée par le client

TABLE 4 – Paquets MQTT observés

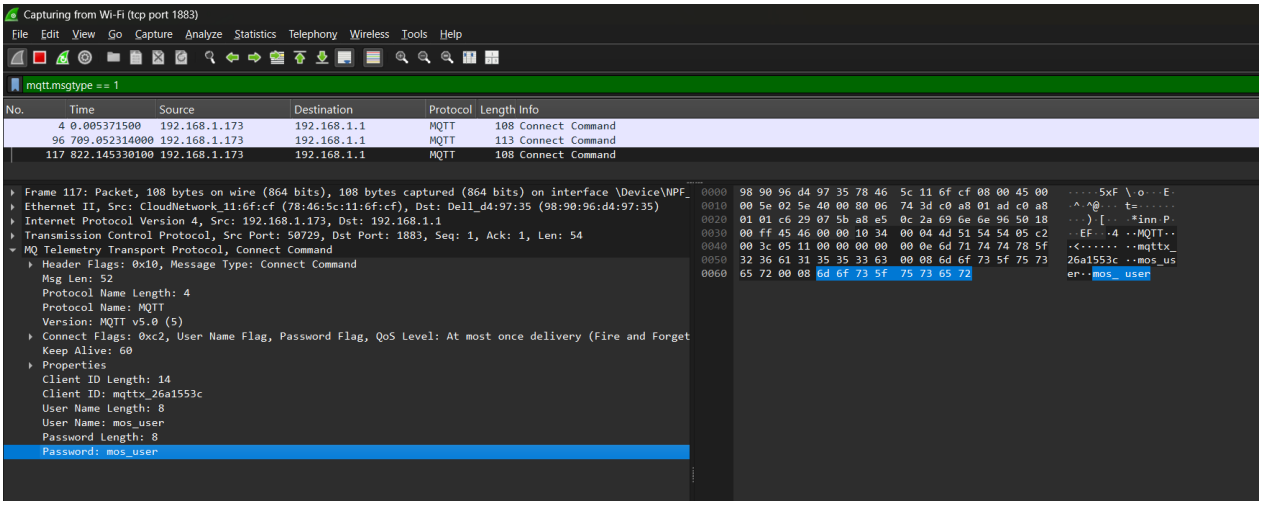


FIGURE 11 – Le mot de passe en clair dans le paquet CONNECT

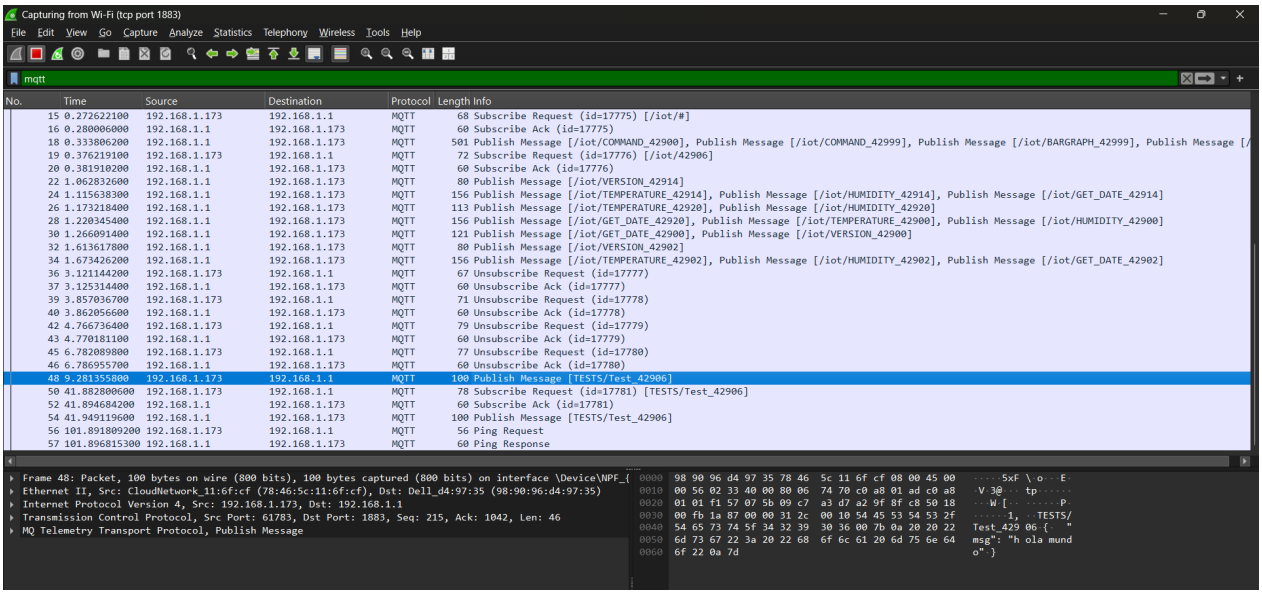


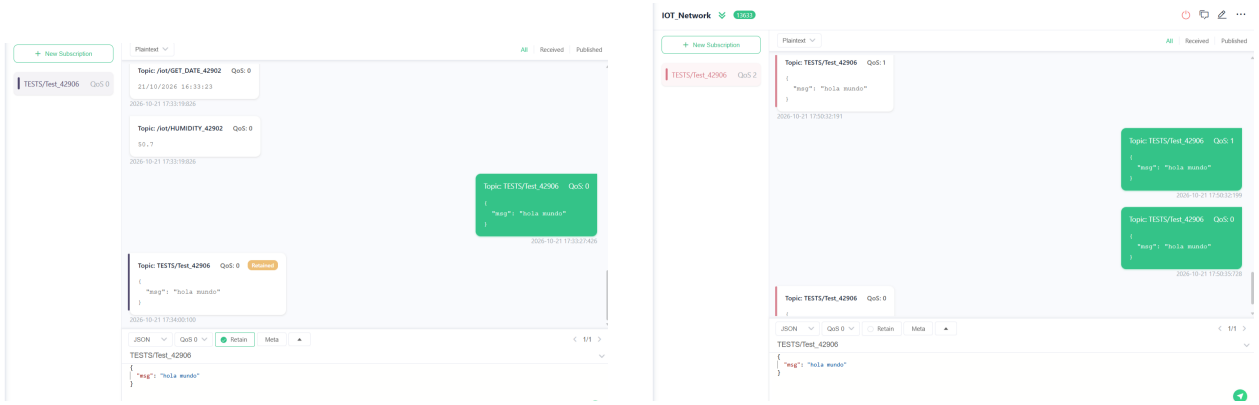
FIGURE 12 – Unsubscribe, puis PINGREQ et PINGRESP pendant une période sans échange

Où apparaît le mot de passe ? Dans le paquet CONNECT, en clair, juste après le nom d'utilisateur. Le port 1883 n'a aucun chiffrement : n'importe qui sur le même WiFi peut lire les identifiants et toutes les données avec Wireshark. Il faudrait passer en MQTT sur TLS (port 8883).

Taille d'un PUBLISH de température. La trame complète fait 85 octets. Si on enlève les en-têtes Ethernet (14), IP (20) et TCP (20), il reste 31 octets pour le message MQTT : 2 octets d'en-tête, 2 pour la longueur du topic, le topic lui-même et la valeur. Une requête HTTP qui enverrait la même valeur porterait plusieurs centaines d'octets d'en-têtes (Host, Content-Type, User-Agent...), et en général une nouvelle connexion TCP à chaque fois. C'est pour ça que MQTT est adapté aux petits objets.

Keep Alive. C'est le temps maximal (60 s ici) pendant lequel le client peut rester sans rien envoyer. S'il n'a rien à publier, il envoie un PINGREQ et le broker répond PINGRESP. Si le broker ne reçoit rien pendant 1,5 fois cette durée, il considère que le client est mort et ferme la connexion.

2.5 QoS, messages retenus et jokers



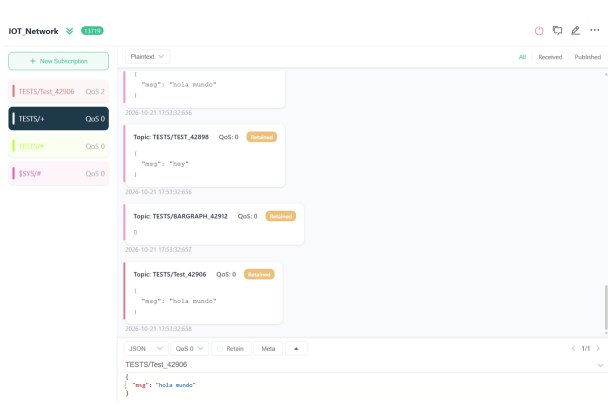
(a) Message retenu reçu après la souscription

(b) Publication en QoS 0, 1 et 2

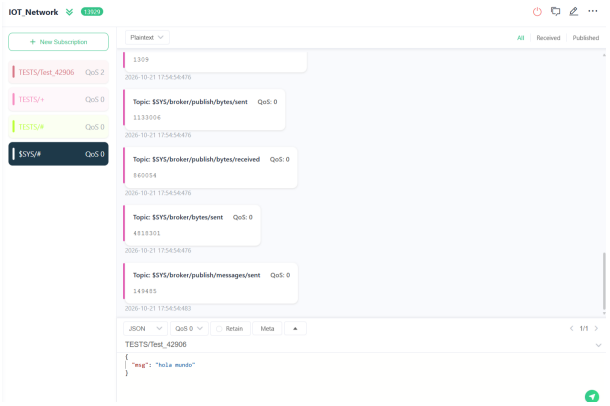
En publiant avec Retain puis en m'abonnant après, je reçois le message tout de suite, marqué « Retained ». Si on publie ensuite un message vide avec Retain, le broker efface le message retenu et les nouveaux abonnés ne reçoivent plus rien.

Pour la QoS, la différence se voit bien dans Wireshark. En QoS 0 il n'y a qu'un PUBLISH. En QoS 1 il y a en plus un PUBACK, et en QoS 2 l'échange complet PUBLISH, PUBREC, PUBREL, PUBCOMP. Plus la garantie de livraison est forte, plus il y a de paquets sur le réseau.

Pour les jokers, `TESTS/+` ne remplace qu'un seul niveau, alors que `TESTS/#` prend tous les niveaux en dessous. Dans les deux cas on reçoit tout de suite les messages retenus des autres postes. Avec `$SYS/#`, on reçoit les statistiques du broker : octets et messages envoyés ou reçus, nombre de clients connectés.



(a) Abonnement `TESTS/+`



(b) Abonnement `$SYS/#`

2.6 Questions générales

1. Acheminement de l'information (modèle TCP/IP). En partant du bas : le boîtier et le PC sont reliés par WiFi 802.11 en 2,4 GHz au point d'accès du routeur RUTX50 (couches physique et liaison). Au niveau réseau, chaque équipement a une adresse IPv4 en 192.168.1.x donnée par le DHCP du routeur, et le broker répond en 192.168.1.1. Au niveau transport, MQTT utilise TCP sur le port 1883, ce qui garantit l'ordre et la livraison des octets, alors que NTP passe par UDP sur le port 123. Enfin, au niveau application, il y a MQTT (publication et abonnement à des topics) et NTP pour l'heure. Une mesure part donc du DHT22, est publiée par l'Arduino, traverse le routeur jusqu'au broker, qui la renvoie à tous les clients abonnés au topic.

2. Services du RUTX50 et place dans l'architecture IoT. Le RUTX50 est un routeur industriel 5G, mais il fait beaucoup plus que router. Dans la section Services du wiki Teltonika, on trouve un broker et un client MQTT, Data to Server (envoi des données vers un serveur), Modbus, OPC UA, BACnet et DNP3 pour les automates, des VPN, SNMP, un Hotspot, du DNS dynamique et même Python3. Il y a aussi les services réseau classiques comme le DHCP et la mise à l'heure NTP. Dans une architecture IoT, je le place au niveau Fog (ou Edge) : il est entre les capteurs et le cloud, et il peut déjà traiter ou filtrer les données sur place. Il peut servir de pont entre le WiFi et l'Ethernet, qui sont dans le même réseau local, et de passerelle entre ce réseau et l'extérieur (5G ou WAN) avec du NAT. Il peut aussi être une passerelle de protocoles, par exemple lire des automates en Modbus et renvoyer les valeurs en MQTT.

3. MCU ou MPU ? C'est un MPU. Les fiches techniques donnent un processeur ARM Cortex-A7 à quatre cœurs à 717 MHz (un Qualcomm IPQ4018), avec 256 Mo de RAM et 256 Mo de flash à côté. Il fait tourner RutOS, un système Linux basé sur OpenWrt. L'Arduino, lui, a un MCU qui exécute un seul programme sans système d'exploitation : il ne pourrait pas gérer en même temps la 5G, le WiFi, le VPN et tous ces services.

4. Les connexions WiFi sont-elles illimitées ? Non. Le RUTX50 est en WiFi 5 (802.11ac, 2,4 et 5 GHz) et les fiches techniques que j'ai trouvées annoncent jusqu'à 150 connexions simultanées. En plus, la bande passante est partagée entre tous les clients et la plage DHCP limite le nombre d'adresses. Le plus gros risque, c'est que le mot de passe du WiFi est le même pour tout le monde (il est même écrit dans le sujet). Une fois connecté, on est sur le même réseau que tous les capteurs et le broker, et comme MQTT n'est pas

chiffré, on peut lire les mesures ou envoyer des commandes à n'importe quel boîtier. La topologie est en étoile : tous les équipements passent par le point d'accès.

5. Mesures de sécurité du firmware. La page « Device security » du wiki Teltonika en donne plusieurs. Le firmware et les paquets sont signés numériquement, et on peut vérifier leur hash SHA-256 avant une mise à jour. Par défaut, l'accès à distance (SSH, WebUI, CLI) est désactivé, la protection contre les attaques SYN flood est active et une adresse IP est bloquée après 10 essais ratés sur SSH ou sur l'interface web. Teltonika fait aussi faire des tests d'intrusion sur RutOS, et son processus de développement est certifié IEC 62443-4-1. Par contre, plusieurs protections restent à activer soi-même (anti scan de ports, attaques ping), et le mot de passe par défaut est imprimé sur l'étiquette. Le fabricant conseille donc de le changer pour un mot de passe d'au moins 12 caractères, d'utiliser HTTPS et SSH, et WPA3 plutôt que WPA2.

6. Logs du broker Mosquitto. Le broker démarre en version 2.1.2 et charge sa configuration. Les lignes « support available » indiquent ce qu'il sait faire (bridge, persistance, TLS, websockets), pas ce qui est activé. Il tourne avec l'utilisateur `mosquitto` et pas en root, ce qui est bien. Les « Restored 0 » montrent qu'il n'y avait rien de sauvegardé (ni messages, ni clients, ni abonnements). Les warnings sont plus intéressants : les fichiers `passwd` et `acl` sont lisibles par tout le monde et n'appartiennent pas à `mosquitto`, ce qui est un risque et sera refusé par les prochaines versions. La règle ACL `TESTS/#` ne contient ni `%c` ni `%u`, donc elle s'applique à tous les clients de la même façon, ce qui explique qu'on puisse lire les topics des autres postes. Enfin, le broker écoute seulement sur le port 1883 (IPv4 et IPv6), sans TLS.

3 Partie 3 : IHM d'acquisition et d'archivage

3.1 Architecture

L'application est en Python, avec `paho-mqtt` pour MQTT, PySide6 pour l'interface, `sqlite3` pour la base et Matplotlib pour les courbes. Les fichiers de `IHM/ihm_mqtt_simple` sont :

Fichier	Rôle
`client_mqtt.py`	classe `ClientMQTT` : connexion, publication, souscription, archivage SQLite
`thread_mqtt.py`	classe `ThreadMQTT` : lit les messages en tâche de fond
`mon_ihm.ui` / `.py`	fenêtre principale dessinée dans Qt Designer, convertie avec `pyside6-uic`
`main.py`	classe `MainWindow` : relie les boutons aux actions
`graphe.ui` / `.py`	boîte de dialogue des courbes
`dialogue_courbe.py`	classe `CourbeDialog` : requête SQLite et tracé Matplotlib
`IOT.db`	base SQLite, créée au premier lancement

TABLE 5 – Fichiers de l'IHM

3.2 Clients MQTT en console

J'ai commencé par deux petits programmes en console qui utilisent la même classe `ClientMQTT`. Le premier s'abonne aux quatre topics du capteur et affiche ce qu'il reçoit. Le deuxième propose un menu pour envoyer les commandes : par exemple, « arrêter les mesures » publie 30 puis 40, pour que la LED clignote lentement et montre l'état du capteur. Les deux tournent en même temps à condition d'avoir des identifiants différents (`Client_42906` et `Client_E42906`), sinon le broker déconnecte le premier.



FIGURE 15 – Programme de lecture connecté au broker

```
Votre choix : 2
Publication : ‘20‘ sur ‘/iot/COMMAND_42906‘
Démarrage des mesures sur le capteur : 42906
Publication : ‘50‘ sur ‘/iot/COMMAND_42906‘
Clignotement rapide de la led de vie du capteur : 42906
```

Ensuite j’ai fusionné les deux avec un thread : `ThreadMQTT` hérite de `Thread`, s’abonne aux topics et appelle `wait_mqtt(timeout=1.0)` en boucle. Le délai de 1 s lui permet de vérifier régulièrement s’il doit s’arrêter. Les valeurs reçues ne sont plus affichées mais stockées dans l’objet `ClientMQTT`, et le menu peut les lire à tout moment.

Un détail qui m’a fait perdre du temps : les exemples du sujet utilisent des topics en minuscules (`temperature`), alors que mon boîtier publie en majuscules (`TEMPERATURE_42906`). MQTT fait la différence entre les deux, donc j’ai tout gardé en majuscules.

3.3 Interface PySide6

La fenêtre a trois zones : les paramètres du broker, l’affichage des données et les commandes. Chaque bouton est relié à une méthode de `MainWindow` par le mécanisme signal / slot de Qt (`clicked.connect(...)`). Le bouton « Connecter » crée le client MQTT avec les valeurs saisies, lance le thread de lecture et démarre un `QTimer` qui rafraîchit l’affichage toutes les secondes. Des `QIntValidator` limitent les saisies (0 à 1023 pour le bargraphe, 100 à 10000 ms pour la période). À la fermeture de la fenêtre, `closeEvent` arrête le thread, sinon le programme resterait ouvert en arrière-plan.



FIGURE 16 – IHM connectée au capteur

Le boîtier réagit bien aux commandes de l'IHM. Par exemple, en envoyant 1023 sur le bargraphe, toutes ses LEDs s'allument :



(a) Avant la commande



(b) Après l'envoi de 1023 sur le bargraphe

3.4 Archivage dans SQLite

La base `IOT.db` contient une table :

```
CREATE TABLE IF NOT EXISTS IOT_DATA
  (Id_capteur integer, Date text, Version text, Temperature real, Humidite real)
```

La connexion SQLite est ouverte au début de `run()` et fermée à la fin, parce qu'un objet SQLite ne peut être utilisé que dans le thread qui l'a créé, et c'est dans ce thread que les messages arrivent. Le sujet propose d'enregistrer à la réception de l'humidité, mais mon boîtier publie la date en dernier : j'enregistre donc la ligne quand `GET_DATE_` arrive, quand les trois valeurs de la mesure sont déjà là.

J'ai aussi changé le format de la date dans la base : elle est stockée en `AAAA-MM-JJ HH:MM:SS`. SQLite compare les dates comme du texte, et seul ce format donne le bon ordre, ce qui sert ensuite pour filtrer avec `BETWEEN`. Enfin, le thread envoie la commande 10 au démarrage, sinon la version n'est jamais publiée (depuis l'étape 6.1.6) et la colonne resterait vide.

```
(42906, '2026-11-04 13:30:30', '1.0', 25.3, 56.8)
(42906, '2026-11-04 13:30:28', '1.0', 25.2, 65.1)
(42906, '2026-11-04 13:30:25', '1.0', 25.1, 70.1)
```

3.5 Affichage des courbes

Le menu « Fichier > Courbes des données » ouvre une boîte de dialogue avec une date de début et une date de fin (par défaut, la dernière heure). « Charger les données » fait la requête `SELECT ... WHERE Date BETWEEN ? AND ?`, trace la température et l'humidité avec Matplotlib, convertit la figure en image (`FigureCanvasAgg` puis `QPixmap`) et l'affiche dans le `QGraphicsView`. Les dates sont converties en `datetime`, ce qui donne un vrai axe de temps : les points sont espacés selon l'heure de la mesure. Sur la figure 18, on voit bien le moment où j'ai soufflé sur le capteur : l'humidité monte de 41 % à 70 % puis redescend.

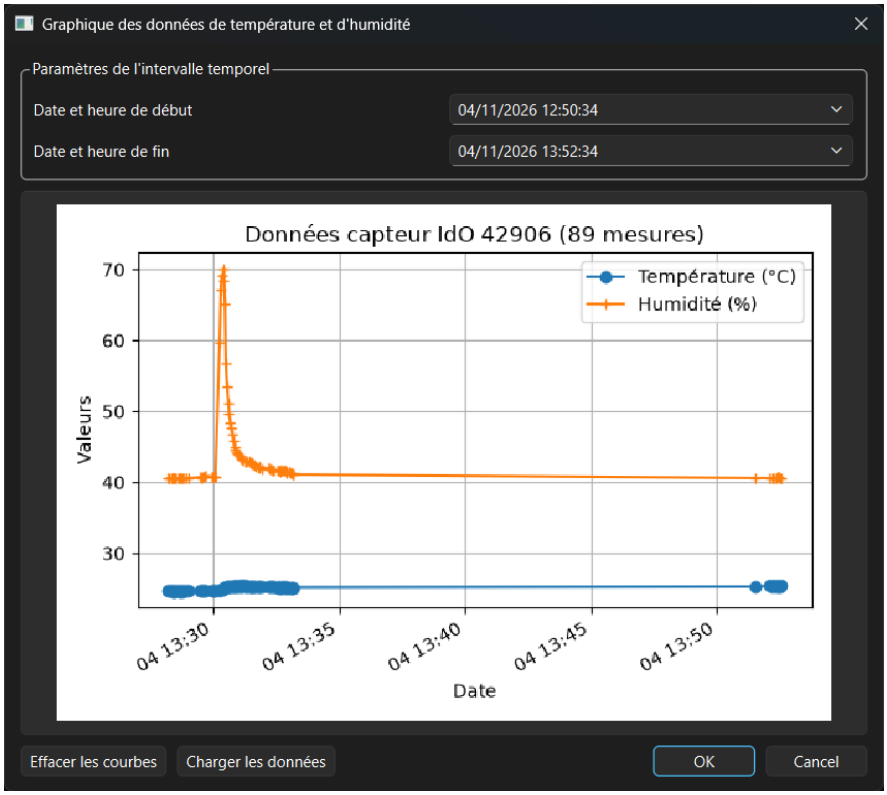


FIGURE 18 – Courbes de température et d'humidité sur 89 mesures

3.6 Notice de fonctionnement

1. Connecter le PC au WiFi « IoT IMT Nord Europe » et brancher le boîtier.
2. Lancer `python IHM/ihm_mqtt_simple/main.py`.

3. Vérifier les paramètres du broker (192.168.1.1, 1883, `mos_user`) et cliquer sur « Connecter ». La barre d'état affiche « Connection au broker MQTT OK ».
4. La date, la température et l'humidité se mettent à jour à chaque nouvelle mesure. Le capteur ne publie que si une valeur change, donc l'affichage peut rester vide quelques secondes au début.
5. Les boutons de la zone Commandes envoient les commandes au boîtier : synchronisation NTP, arrêt ou démarrage des mesures, clignotement lent, rapide ou personnalisé (période en ms, multiple de 100). Pour le bargraphe, saisir une valeur entre 0 et 1023 puis appuyer sur Entrée ou sur le bouton.
6. Pour voir l'historique : « Fichier > Courbes des données », choisir l'intervalle et cliquer sur « Charger les données ». Cette partie lit seulement la base, elle marche même sans connexion.
7. « Déconnecter » arrête la lecture. On peut aussi fermer la fenêtre directement, le thread s'arrête proprement.

3.7 Le challenge

Je n'ai pas fait le challenge (lire les capteurs de toute la salle). Il demande que chaque poste modifie son programme Arduino pour publier un topic `DATA_` sérialisé avec le caractère « | », et il n'y avait pas assez de postes prêts pour le tester. Le code est quand même préparé pour ça : l'identifiant du capteur est déjà extrait du topic et enregistré dans la base.

Conclusion

Ce projet m'a fait parcourir toute la chaîne d'un objet connecté : le circuit et le programme embarqué, le réseau et le protocole, puis l'application qui exploite les données. La simulation m'a permis de valider la logique avant de toucher au matériel. Avec Wireshark, j'ai vu concrètement ce qu'il y a dans les trames MQTT et à quel point le protocole est léger, mais aussi qu'il n'est pas du tout sécurisé sur le port 1883.

Les difficultés venaient surtout de détails de configuration plus que du code : le réseau IoT sans Internet (il fallait tout installer avant et trouver le serveur NTP local), le changement d'heure, le WiFi 5 GHz que l'UNO R4 ne voit pas, la casse des topics, ou le format de date pour SQLite. Travailler par petites étapes avec un commit à chaque fois m'a beaucoup aidé à trouver d'où venaient les erreurs.

Pour aller plus loin, il faudrait chiffrer les échanges (MQTT sur TLS, port 8883) et donner à chaque capteur des droits seulement sur ses propres topics. On pourrait aussi faire le challenge avec une publication sérialisée et retenue, et mettre les paramètres du broker dans un fichier de configuration au lieu de les écrire dans le code.